

Aprendizaje Supervisado

A continuación, estudiaremos tres de los algoritmos de aprendizaje supervisado más utilizados: Árboles de Decisión, Máquina de Vectores de Soporte y Navie Bayes.

Árboles de Decisión (Decision Tree)

El árbol de decisión es un grafo que representa elecciones y sus resultados en forma de árbol. Los nodos del grafo representan un evento o elección y las ramas o aristas del gráfico representan las reglas o condiciones de decisión. Los árboles de decisión se pueden aplicar tanto a problemas de regresión como de clasificación e implican la estratificación o segmentación del espacio predictor en varias regiones simples (Suthaharan, 2016). Para los árboles de regresión, estos atribuyen valores a nuevos nodos dividiendo los datos en subconjuntos. Por otro lado, los árboles de clasificación otorgan etiquetas categóricas a nuevos nodos segmentando los datos en subconjuntos y estableciendo la categoría más frecuente en cada nodo. Los dos modelos basados en árboles son sencillos de entender y pueden gestionar diversos tipos de datos, lo que los hace herramientas adaptables tanto para el análisis predictivo como para entender relaciones complejas en los datos.

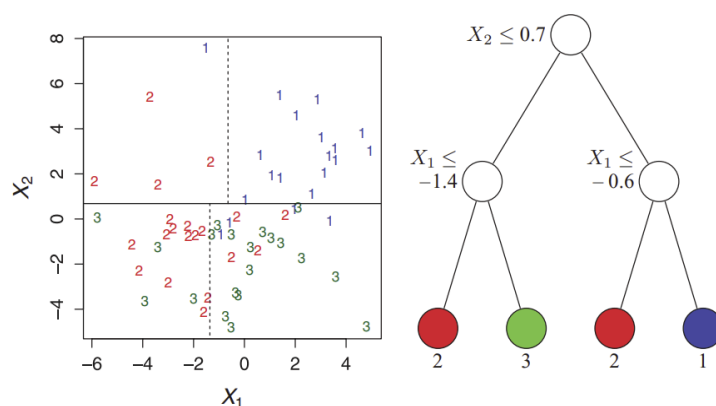


Figura 1. Particiones (izquierda) y estructura de un árbol de decisión para clasificación para tres clases: 1,2 y 3. Fuente: Loh, W.-Y. (2011).

Formulación

Una formulación matemática rigurosa de los árboles de decisión la podemos encontrar en la documentación de Scikitlearn¹. Dado los vectores de entrenamiento $x_i \in R^n$, $i = 1, \dots, l$ y un vector de etiquetas $y \in R^l$, representaremos a los datos en el nodo m como Q_m con n_m muestras. Para cada división de candidatos $\theta = (j, t_m)$ compuesto por un atributo j y un umbral t_m , la partición de lo datos en $Q_m^{left}(\theta)$ y $Q_m^{right}(\theta)$ subconjuntos viene dado:

¹ <https://scikit-learn.org/1.5/modules/tree.html>

$$Q_m^{left}(\theta) = \{(x, y) | x_j \leq t_m\}$$

$$Q_m^{right}(\theta) = Q_m \setminus Q_m^{left}(\theta)$$

La calidad de una división de candidatos de un nodo m es entonces calculada usando una función de impureza (impurity function) o función de pérdida $H()$ dependiendo de si es para clasificación o regresión y viene dado por:

$$G(Q_m, \theta) = \frac{n_m^{left}}{n_m} H(Q_m^{left}(\theta)) + \frac{n_m^{right}}{n_m} H(Q_m^{right}(\theta))$$

Seleccionando los parámetros que minimizan la impureza o pérdida

$$\theta^* = \operatorname{argmin}_{\theta} G(Q_m, \theta)$$

Este proceso se hará de manera recursiva hasta que $n_m < \min_{samples}$ o $n_m = 1$

Navie Bayes

Es una técnica de clasificación basada en el teorema de Bayes con un supuesto de independencia entre los predictores. En términos simples, un clasificador Naive Bayes supone que la presencia de un atributo particular en una clase no está relacionada con la presencia de ningún otro atributo. El clasificador Naive Bayes se utiliza mucho en la clasificación de texto, por ejemplo, para asignar temas en el texto, detectar spam, identificar edad/género a partir del texto y realizar análisis de sentimientos.

Formulación

De acuerdo a Kamperis (2020), dado un set de atributos $x_1, x_2, \dots, x_n$ y un set de clases C , se quiere construir un modelo que devuelva un valor de $P(C_k | x_1, x_2, \dots, x_n)$. Por tanto, si se toma la probabilidad máxima sobre este rango de probabilidades se puede obtener la mejor estimación de la clase correcta tal que:

$$\begin{aligned} C_{predicted} &= \operatorname{argmax}_{c_k \in C} P(C_k | x_1, x_2, \dots, x_n) \\ &= \operatorname{argmax}_{c_k \in C} \frac{P(x_1, x_2, \dots, x_n | C_k) P(C_k)}{P(x_1, x_2, \dots, x_n)} \end{aligned}$$

Asumiendo que los atributos $x_1, x_2, \dots, x_n$ son independientes entonces:

$$P(C_k | x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i | C_k).$$

El modelo se simplifica de tal manera que:

$$C_{predicted} = P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

Máquina de Vectores de Soporte (SVM)

Las máquinas de vectores de soporte (SVM por su siglas en inglés) son modelos de aprendizaje supervisados cuyos algoritmos de aprendizaje analizan los datos utilizados para la clasificación y el análisis de regresión. Además de realizar una clasificación lineal, las SVM pueden realizar una clasificación no lineal de manera eficiente utilizando el llamado truco del kernel, mapeando implícitamente sus entradas en espacios de atributos de alta dimensión. Básicamente, dibuja márgenes entre clases (hiperplano). Los márgenes se trazan de tal manera que la distancia entre el margen y las clases (vectores de soporte) sea máxima y, por tanto, minimice el error de clasificación (ver figura 2).

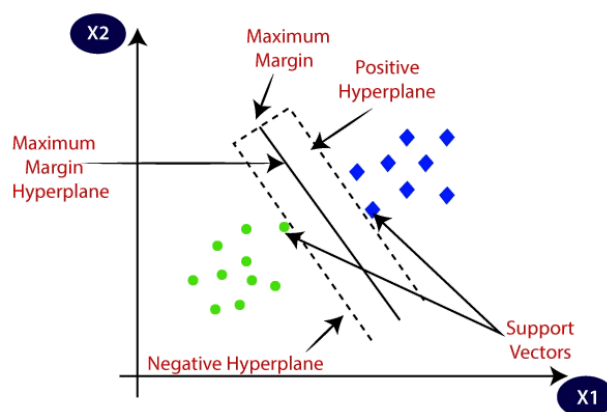


Figura 2. Representación gráfica de los términos más importantes del algoritmo SVM y cómo funciona. Fuente: <https://www.analyticsvidhya.com/blog/2021/10/support-vector-machinessvm-a-complete-guide-for-beginners/>

Las SVM son altamente adaptables, lo que las hace adecuadas para diversas aplicaciones, como clasificación de texto, clasificación de imágenes, detección de spam, identificación de escritura a mano, análisis de expresión genética, detección de rostros y detección de anomalías.

Formulación Matemática

Consideremos un conjunto de datos de entrenamiento con **X** como matriz de atributos (cada fila es un punto de datos). También considerar **y** como vector de etiquetas (1 o -1 para cada clase). Se quiere encontrar un hiperplano que se defina como (Saini, A. 2021):

$$w^t \cdot X + b = 0$$

Donde: **w**: Vector de pesos (normal al hiperplano) y **b**: Bias (intercepto).

Se quiere maximizar el margen entre las clases. El margen se define como la distancia perpendicular desde el hiperplano al punto de datos más cercano de cada clase.

$$\operatorname{argmax}(w^*, b^*) = \frac{2}{||w||}$$

Sujeto a:

$$y_i(\bar{w} \cdot \bar{X} + b) \geq 1$$

Con lo que se asegura que los puntos de datos estén correctamente clasificados y a una distancia mayor o igual a 1 del hiperplano (margen)

Aprendizaje No Supervisado

Los algoritmos de aprendizaje no supervisados aprenden pocas características de los datos. Cuando se introducen nuevos datos, utilizan características aprendidas previamente para reconocer la clase de los datos. Se utilizan principalmente para agrupación y reducción de características (Naeem, S. et al. 2023).

Agrupamiento de K-Medias (K-Means clustering)

El procedimiento sigue un método sencillo que clasifica un conjunto de datos determinados en varios grupos. La idea principal es definir K-centros (o centroides), uno para cada grupo. Diferentes ubicaciones de centroides conducen a resultados diferentes, por tanto, la mejor opción es colocarlos lo más lejos posible uno del otro (Kodinariya, T. & Makwana, P. 2013).

El siguiente paso es tomar cada punto perteneciente a un conjunto de datos determinado y asociarlo con el centroide más cercano. Cuando no quedan puntos pendientes, se completa la primera etapa y se pasa a recalcular los K nuevos centroides como el centroide de los conglomerados resultantes del paso anterior (figura 3).

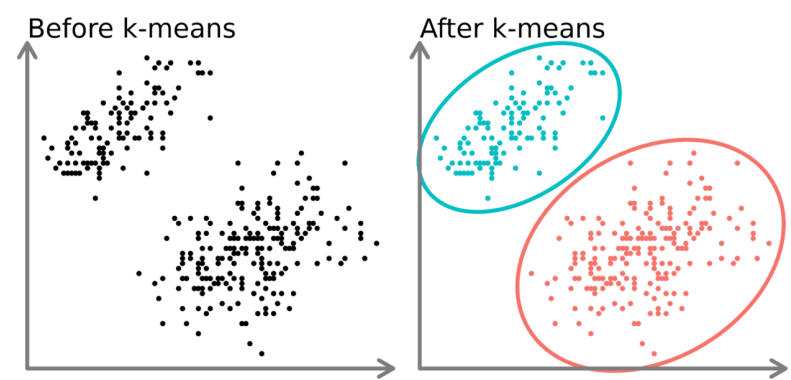


Figura 3. Técnica de aprendizaje no supervisado mediante método de k-medias. Fuente: <https://www.linkedin.com/pulse/k-means-clustering-applications-real-world-mz4df/>

En algunos casos, K no está claramente definido y debemos pensar en el número óptimo de estos. La agrupación de K-Medias funciona mejor cuando los datos están bien separados. Este método se utiliza

ampliamente para la compresión de imágenes, motores de recomendación, agrupación de documentos y segmentación de clientes.

Formulación Matemática

Este algoritmo tiene como objetivo minimizar una función objetivo, en este caso una función de error al cuadrado. La función objetivo (Sinaga, K. & Yang, M. (2020):

$$w(S, C) = \sum_{k=1}^K \sum_{i \in S_k} \|y_i - c_k\|^2$$

Donde S es una partición de grupo K de atributos representadas por los vectores y_i en el espacio de atributos de dimensión M, que consta de S_k grupos no vacíos y que no se superponen, cada uno con un centroide c_k ($k = 1, 2, \dots, K$).

Máquina de Boltzmann restringida (RBM)

Las máquinas de Boltzmann restringidas (RBM por su siglas en inglés) se utilizan normalmente para el aprendizaje no supervisado. Este método toma un conjunto de datos de entrada y los divide en dos partes: la capa visible y la capa oculta. La capa visible está compuesta por neuronas conectadas al conjunto de datos original, mientras que la capa oculta está compuesta por neuronas que no están conectadas al conjunto de datos original. El objetivo de este algoritmo es aprender la relación entre las capas de entrada y salida. Debido a la flexibilidad de los RBM, desempeñan un papel vital en diversas aplicaciones, como la reducción de dimensionalidad, el filtrado colaborativo, el modelado de temas (topic-modeling), la clasificación y el aprendizaje de atributos (feature learning).

Formulación matemática

Al tratarse de un método basado en una red neuronal su formulación detallada puede resultar compleja, lo que excede el alcance de este trabajo. Sin embargo, podemos resumirlo en que este método aprende una distribución de datos de probabilidad utilizando un conjunto de muestras de distribución, llamados datos de entrenamiento. Durante el entrenamiento, los parámetros de RBM, los pesos W_{ij} y los sesgos a_i, b_j , se ajustan para que la distribución marginal de los giros visibles $p(v)$ coincida con la distribución de datos representada por el entrenamiento del conjunto de datos. Después de un entrenamiento exitoso, el muestreo de la capa visible de RBM debería (idealmente) producir nuevas muestras que también provengan de la misma distribución subyacente de los datos de entrenamiento.

Una forma de determinar la distribución de probabilidad más cercana a la de entrenamiento se puede obtener a través de la divergencia Kullback–Leibler²:

$$O = \sum_v q(v) \log\left(\frac{q(v)}{p(v)}\right)$$

Donde O es la entropía relativa de las distribuciones, $p(v)$ es la distribución marginal de RBM y $q(v)$ es la distribución de los datos. Los valores óptimos de W_{ij} , a_i y b_j se obtienen mediante descenso del gradiente:

$$W_{ij} \rightarrow W_{ij} - \eta \frac{\partial O}{\partial W_{ij}}$$

$$a_i \rightarrow a_i - \eta \frac{\partial O}{\partial a_i}$$

$$b_j \rightarrow b_j - \eta \frac{\partial O}{\partial b_j}$$

Donde η es la tasa de aprendizaje.

Codificador automático (Autoencoder)

Se trata de un tipo de red neuronal que se utiliza para el aprendizaje no supervisado. Funciona tomando un conjunto de datos de entrada que son codificados en una capa oculta (Li, P. & Li, J. 2023). Estos luego se decodifican y se comparan con el conjunto de datos de entrada original. Si hay un alto grado de similitud entre los dos conjuntos, el codificador ha hecho su trabajo correctamente. De lo contrario, el codificador deberá ajustarse hasta que exista un alto grado de similitud entre los dos conjuntos. Los codificadores automáticos se utilizan ampliamente en la clasificación y reconstrucción de imágenes.

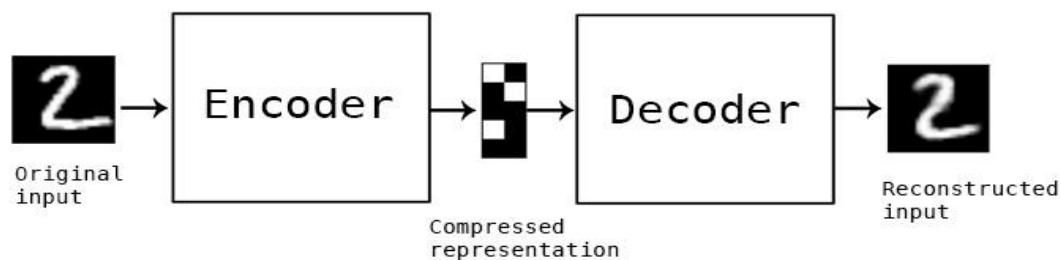


Figura 4. Funcionamiento del algoritmo de codificador automático. Fuente:

<https://www.analyticsvidhya.com/blog/2021/06/complete-guide-on-how-to-use-autoencoders-in-python/>

² <https://courses.physics.illinois.edu/phys498cmp/sp2022/ML/RBM.html>

REFERENCES

- Gareth, J., Daniela, W., Trevor, H., & Robert, T. (2013). An introduction to statistical learning: with applications in R. Springer.
- Kamperis, S. (2020). How to implement a naive Bayes classifier with tensorflow. Let's talk about science!
<https://ekamperi.github.io/machine%20learning/2020/12/31/naive-bayes-classifier-in-tensorflow.html>
- Khanday, N. Y., & Sofi, S. A. (2021). Taxonomy, state-of-the-art, challenges and applications of visual understanding: A review. Computer Science Review, 40, 100374.
- Kodinariya, T. M., & Makwana, P. R. (2013). Review on determining the number of Clusters in K-Means Clustering. International Journal, 1(6), 90-95.
- Li, P., Pei, Y., & Li, J. (2023). A comprehensive survey on design and application of autoencoders in deep learning. Applied Soft Computing, 138, 110176.
- Loh, W. Y. (2011). Classification and regression trees. Wiley interdisciplinary reviews: data mining and knowledge discovery, 1(1), 14-23.
- Naeem, S., Ali, A., Anam, S., & Ahmed, M. M. (2023). An unsupervised machine learning algorithm: Comprehensive review. International Journal of Computing and Digital Systems.
- Saini, A. (2021). Guide on Support Vector Machine (SVM) Algorithm. Analytics Vidhya
- Sinaga, K. P., & Yang, M. S. (2020). Unsupervised K-Means Clustering Algorithm. IEEE Access, 8, 80716-80727. <https://doi.org/10.1109/ACCESS.2020.2988796>
- Suthaharan, S. (2016). Decision tree learning. Machine Learning Models and Algorithms for Big Data Classification: Thinking with Examples for Effective Learning, 237-269.
- Suthaharan, S. (2016). Support Vector Machine. In: Machine Learning Models and Algorithms for Big Data Classification. Integrated Series in Information Systems, vol 36. Springer, Boston, MA. https://doi.org/10.1007/978-1-4899-7641-3_9
- Zhang, H. (2004). The optimality of naive Bayes. Aa, 1(2), 3.

